# Goal

The goal of this lab session is to work with different types of memory, how it's accessed, and the effect these different types have on the performance of the program. We specifically look into coalesced vs uncoalesced memory access and 3 different types of GPU memory.

# (un)coalesced memory access

When a thread requires data from the memory, the GPU not only reads the required memory but reads a chunk of memory, so not only the required memory is read but also some memory around it. When we are talking about uncoalesced memory access we mean that the threads in a warp each need a different part of memory, so multiples of these chunks will be read. When we use coalesced memory access the first thread will read a chunk of memory that includes the other threads required data, not all the other threads will now have to make a memory read as the required memory location could be included in the previously read memory chunk. By optimizing how the data is stored, coalesced memory access could be used to reduce the number of memory reads, as memory reads are time intensive the overall execution time is expected to be lower. To test this theory we preformed a test to compare uncoalesced and coalesced memory access. For uncoalesced memory access an array is used where every fourth element an operation is performed. For Coalesced the data is placed at the beginning of the array, so the memory can be accessed in group. To improve the results an average time was taken for 1000 runs and each kernel function was created to have an equal complexity.

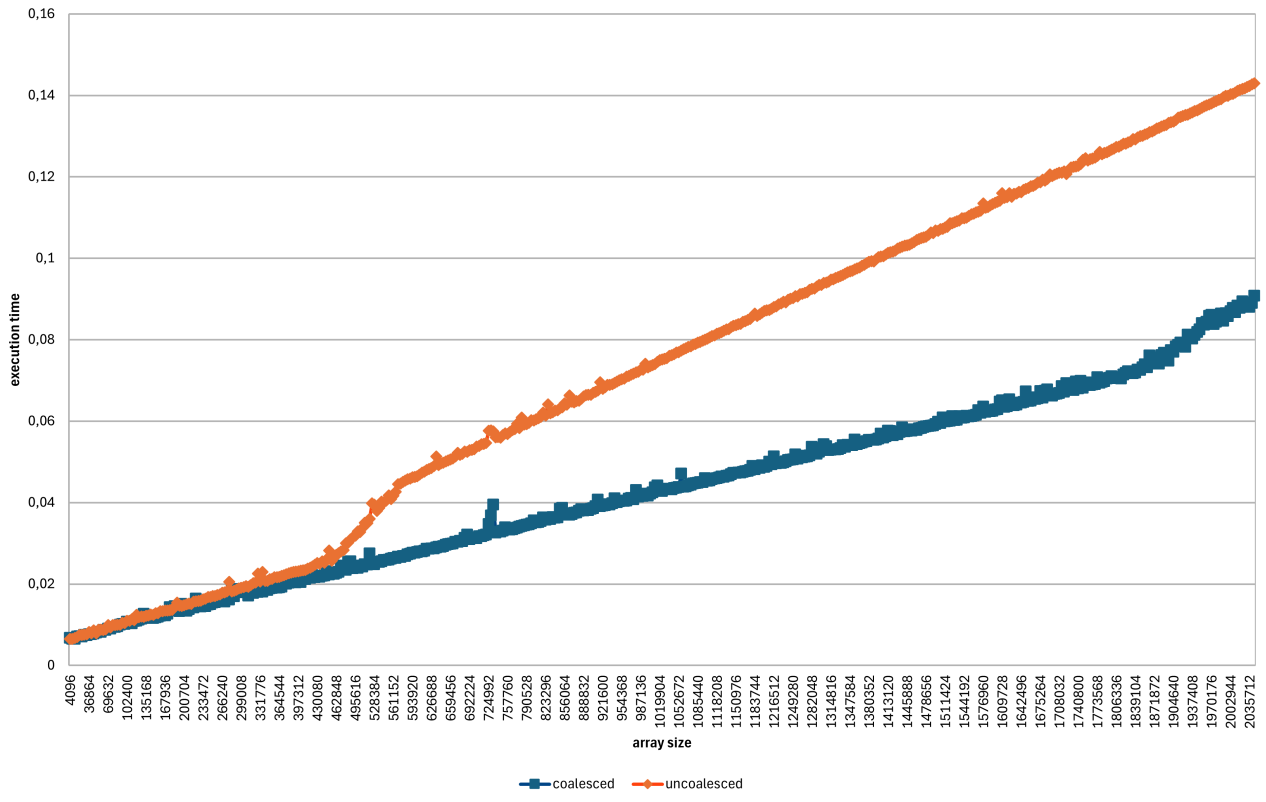



Figure 1: coalesced vs uncoalesced

Figure 1 shows the execution time for coalesced and uncoalesced memory access for varying array size. We can see that for array sizes larger than 50000 the coalesced memory access shows a significant gain in performance of up to 80%.

## Global, shared and constant memory

On a GPU there are many different types of memory. In this section, we compare the performance of 3 of these memories. The first is Global memory that can be accessed by CPU and each thread in a grid. The downside to global memory is the access latency. Shared memory can only be accessed by threads in a block but provides lower access latency. Constant memory is similar to global memory as they both allow access from CPU and the threads in a grid but constant memory cannot be modified from within the kernel [1]. To test the performance of these different memories a multiplication of 2 arrays was performed each using a different memory type. For each size the computation was run for 1000 times. When calculating we have to consider the maximum memory size of 48kB for shared and 64kB for constant memory, in this test these limits were not reached. To use the different memory's the kernel each differ on how they read and store the required memory.

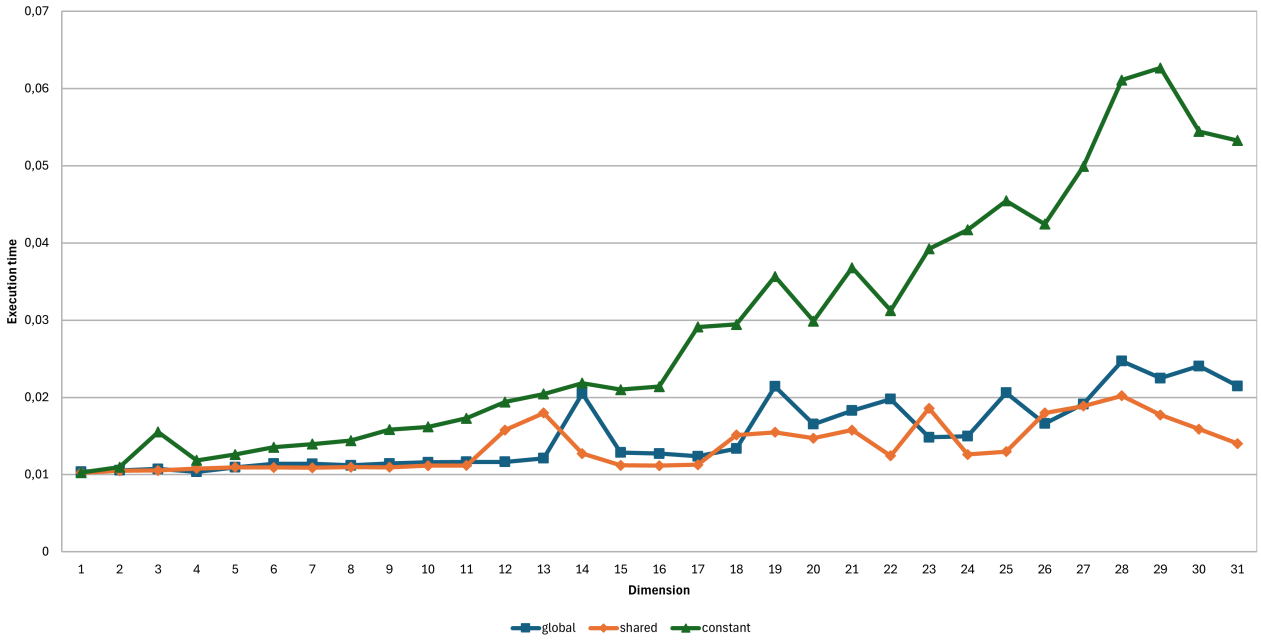



Figure 2: Array multiplication

In figure 2 the execution time to calculate the multiplication is given for varying dimensions and memory types. The figure shows that constant memory is slower compared to the global and shared memory. The global and shared memory look similar at first but when comparing the average time of 0,015230865 to 0,0135978 we can assume that the shared memory is faster. To improve these results, a larger array size should increase the time difference as more memory has to be read. In this test we can conclude that shared memory is faster when comparing these 3 memory's.

## Conclusion

We can conclude that the type of memory used and how data is stored significantly impacts performance, especially when working with large array size's.

[1] https://www.3dgep.com/cuda-memory-model/